Submission Worksheet

Submission Data

Course: IT202-008-S2026
Assignment: IT202 Milestone 1 - 2026
Student: Angel V. (av847)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 4/12/2026 8:32:59 PM
Updated: 4/13/2026 2:52:31 AM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/grade/av847
View Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/av847

Instructions

- Overview Link: https://youtu.be/IDtqdaLwcVg

1. Refer to Milestone1 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `qa`
    5. On Github create a pull request from `qa` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

To Register A New Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

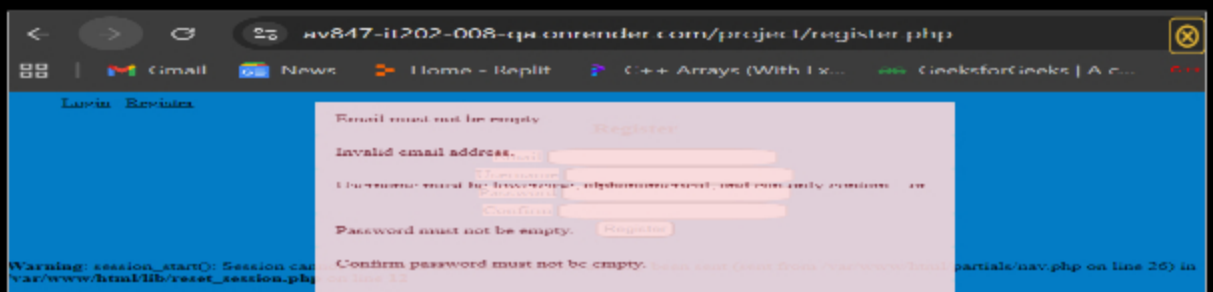
Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
<h3>Register</h3>
<form onsubmit="return validate(this)" method="POST">
    <div>
        <label for="email">Email</label>
        <input id="email" type="email" name="email" required />
    </div>
    <div>
        <label for="username">Username</label>
        <input type="text" name="username" required maxlength="30" />
    </div>
    <div>
        <label for="pw">Password</label>
        <input type="password" id="pw" name="password" required minlength="8" />
    </div>
    <div>
        <label for="confirm">Confirm</label>
        <input type="password" name="confirm" required minlength="8" />
    </div>
    <input type="submit" value="Register" />
</form>
```

HTML validations



HTML validations examples

Saved: 4/12/2026 10:39:41 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

For the validation step, the program is simply set to check individually if the email, username, password, and confirm password meets the proper criteria. This is done through if stamements for each field. If it doesn't meet a certain criteria, then the flash message is triggered.

Saved: 4/12/2026 10:39:41 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

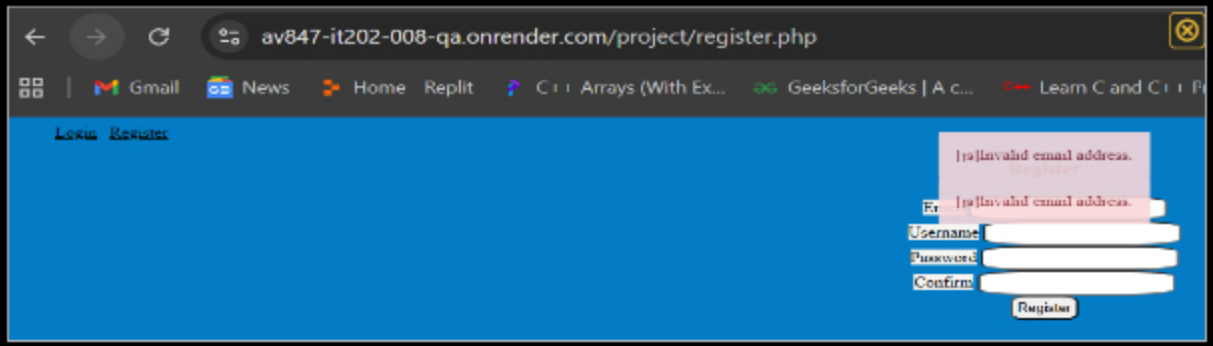
Progress: 100%

Part 1:

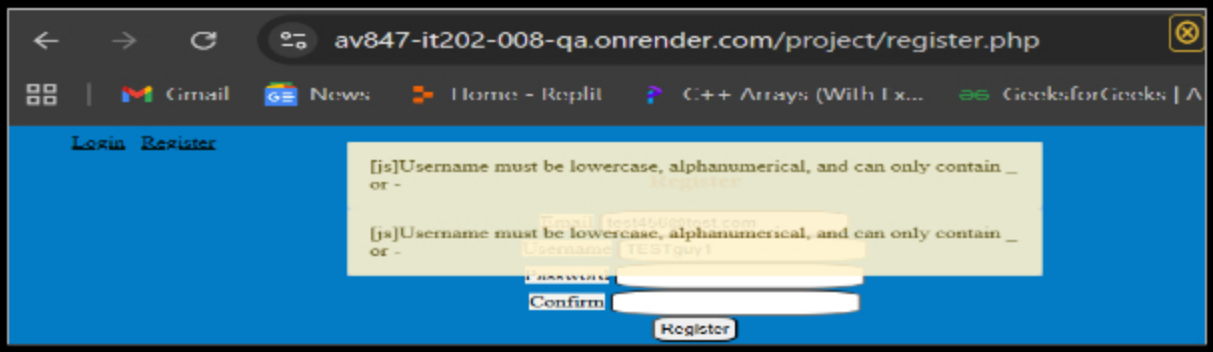
Progress: 100%

Details:

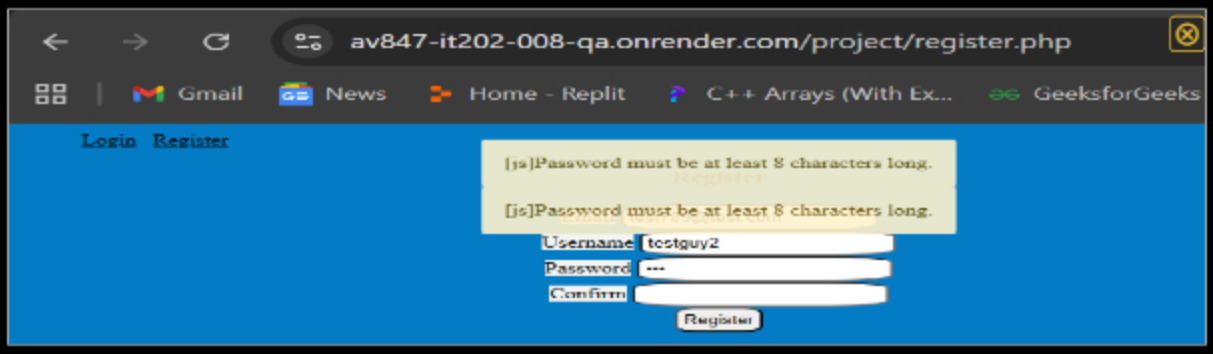
- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



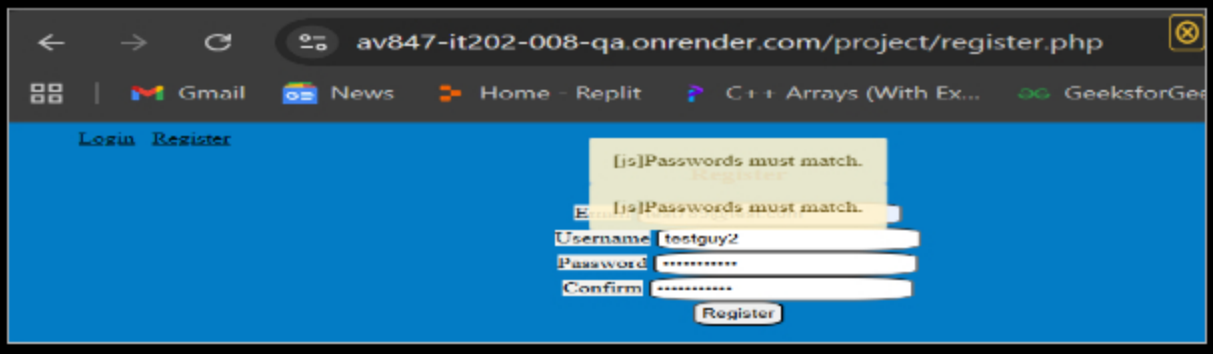
Email validation JS



Username validation JS



Password validation JS



Password and confirm must match JS

Saved: 4/12/2026 10:09:40 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validations for JS was done by again using if statements to validate each field individually. First we check for email, then Username, then password, and finally confirm password. If any of the fields fail to meet the requirements needed to succesfully register, then a flash message pops up on the screen accordingly.

Saved: 4/12/2026 10:09:40 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

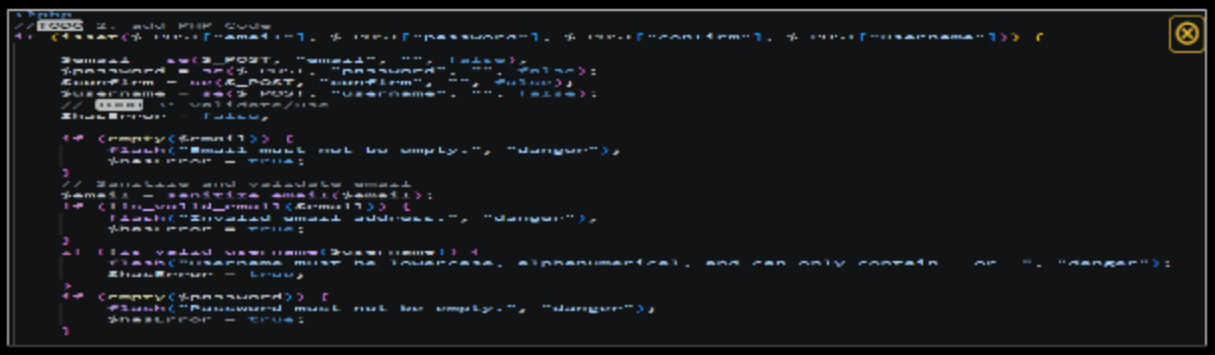
Progress: 100%

Part 1:

Progress: 100%

Details:

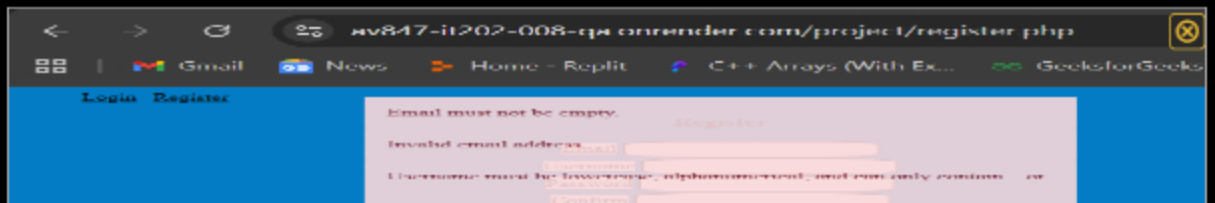
- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

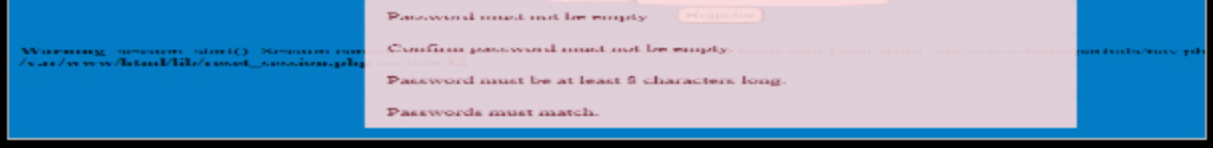


Register page PHP validation (part 1)

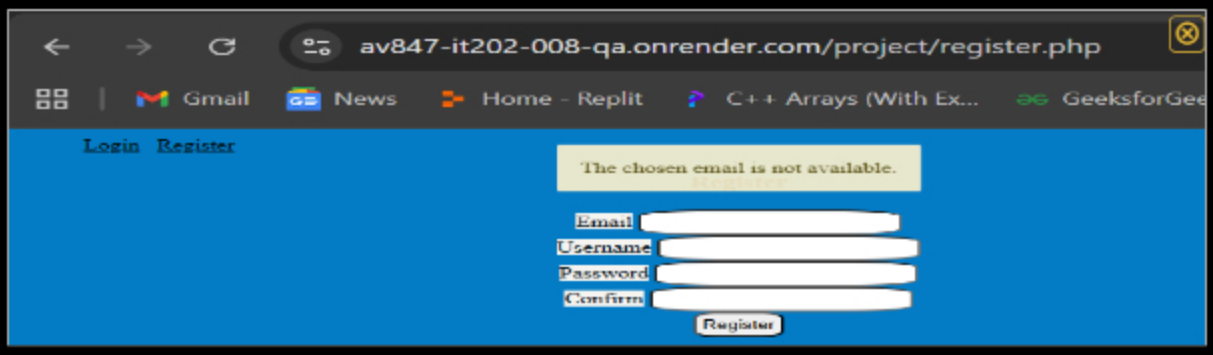
```php
if (empty($confirm)) {
    flash("Confirm password must not be empty.", "danger");
    $hasError = true;
}

if (!is_valid_password($password)) {
    flash("Password must be at least 8 characters long.", "danger");
    $hasError = true;
}

if (!is_valid_confirm($password, $confirm)) {
    flash("Passwords must match.", "danger");
    $hasError = true;
}
```

Register page PHP validation (part 1)

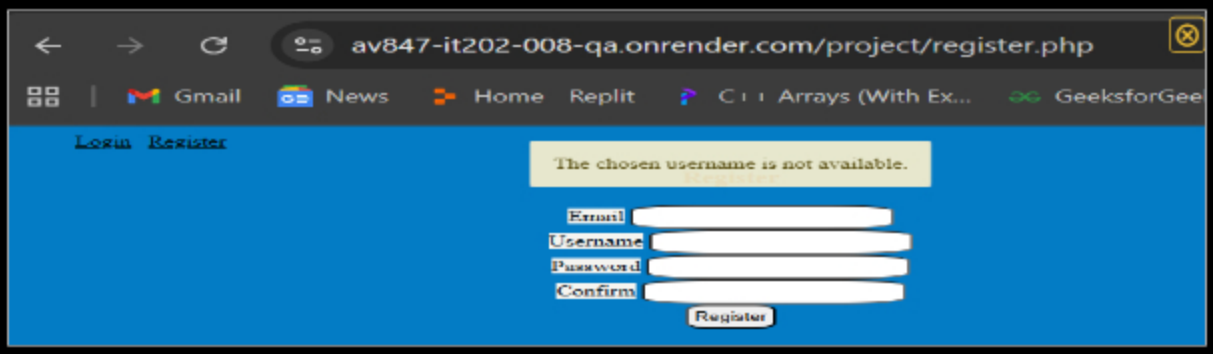




PHP validations



Email taken validation



Username taken validation

Saved: 4/12/2026 10:19:56 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validation steps here are the same as I've previously stated. First we check for email, then Username, then password, and finally confirm password. If any of the fields fail to meet the requirements needed to succesfully register, then a flash message pops up on the screen accordingly.

Saved: 4/12/2026 10:19:56 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php`, , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/av847-NJIT/av847-IT202-008/Milestone1/public_html/project/register.php

URL
https://github.com/av847-NJ

URL #2
https://av847-it202-008-prod.onrender.com/project/register.php

URL
https://av847-it202-008-prod.

URL #3
https://github.com/av847-NJIT/av847-IT202-008/compare/qa...Feat-MS1-BasicNav-Login

URL
https://github.com/av847-NJ

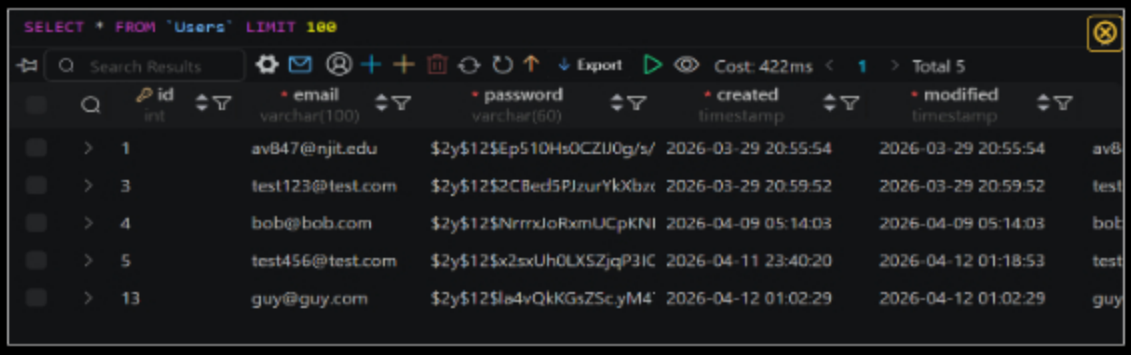
Saved: 4/12/2026 10:29:06 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



Valid registers users

Saved: 4/12/2026 10:30:41 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

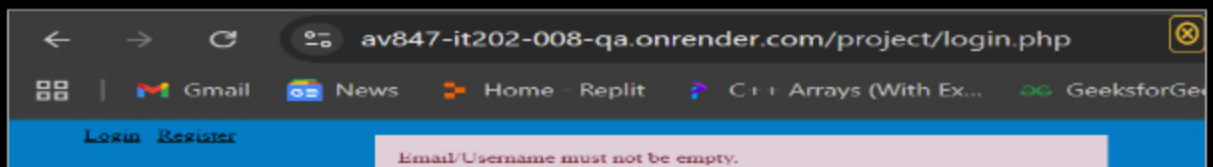
Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
<h3>Login</h3>
<form onsubmit="return validate(this)" method="POST">
    <div>
        <label for="email">Email or Username</label>
        <input id="email" type="text" name="email" required />
    </div>
    <div>
        <label for="pw">Password</label>
        <input type="password" id="pw" name="password" required minlength="8" />
    </div>
    <input type="submit" value="Login" />
</form>
```

Login page form



Login validations

Saved: 4/12/2026 10:41:39 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validations here simply check for email/username and password to meet the requirments. These requirements include that the email or username isn't empty and that the password be over 8 characters long.

Saved: 4/12/2026 10:41:39 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

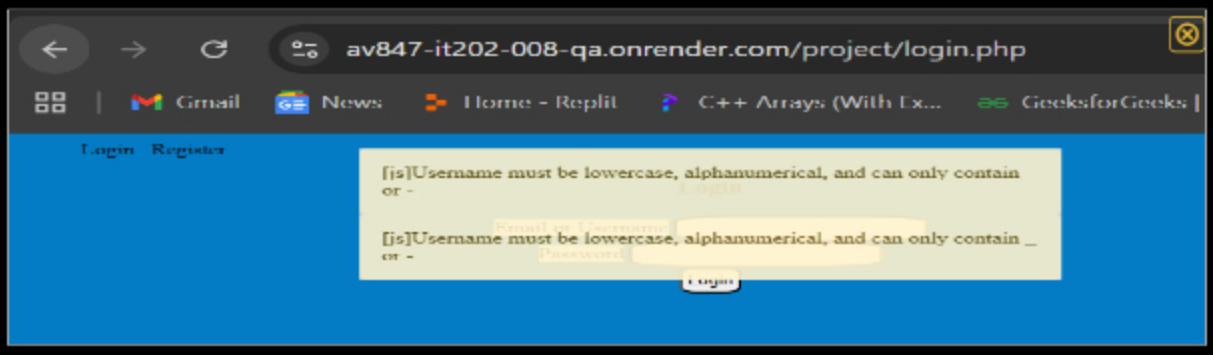
Progress: 100%

Details:

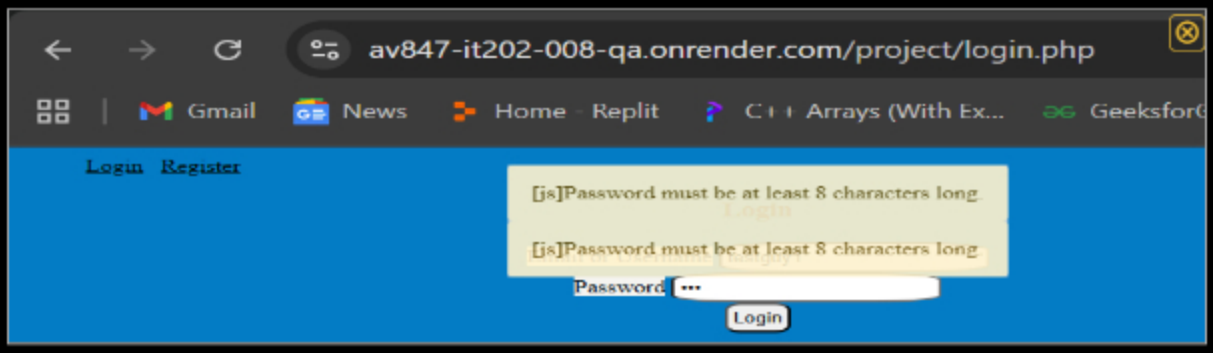
- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```
<script>
    function validate(form) {
        //implement JavaScript validation (you'll do this on your own towards the end of Milestone1)
        //ensure it returns false for an error and true for success
        let email = form.email.value.trim();
        let password = form.password.value;

        if (email.includes("@")) {
            if (!isValidEmail(email)) {
                flash("Invalid email address.", "warning");
                return false;
            }
        } else {
            if (!isValidUsername(email)) {
                flash("Username must be lowercase, alphanumerical, and can only contain _ or -", "warning");
                return false;
            }
        } <!-- #30-35 else

        if (!isValidPassword(password)) {
            flash("Password must be at least 8 characters long.", "warning");
            return false;
        }

        return true;
    } <!-- #19-42 function validate(form)
```

Code for js validations



Email and Username validations JS



Password validations JS

Saved: 4/12/2026 10:54:08 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validations here work by first checking if the email entered includes an @ symbol. If it does, it then checks if the email is valid via the regex requirements on the function in helpers.js. If it is not, then a falsh message appears accordingly. Then, the program proceeds to check if the username and password are valid through conditions as well. If neither of those are valid, then the appropriate flash messages are triggered.

Saved: 4/12/2026 10:54:08 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

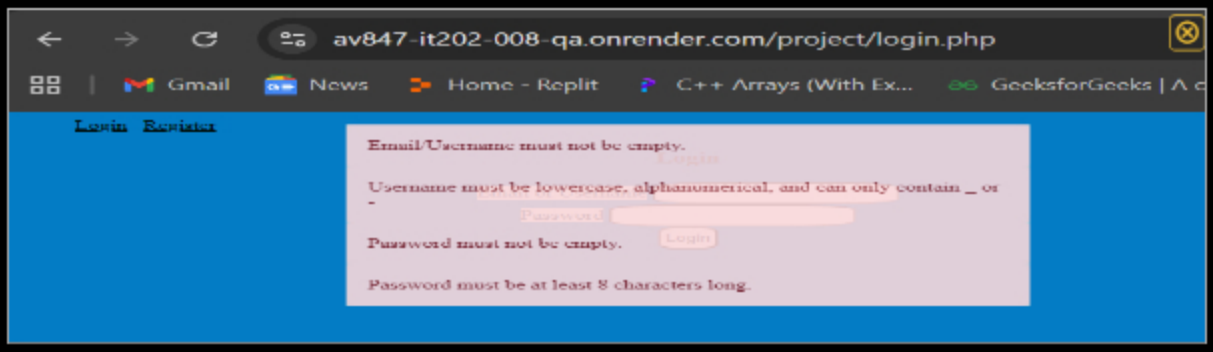
Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)

php validation (part 1)

```php
if (empty($password)) {
    flash("Password must not be empty.", "danger");
    $hasError = true;
}

if (!is_valid_password($password)) {
    //echo "Password too short<br>";
    flash("Password must be at least 8 characters long.", "danger");
    $hasError = true;
} <- #83-87 if (!is_valid_password($password))
```

php validations (part 2)



Validations messages

Saved: 4/12/2026 11:02:23 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validation steps here begin by establishing a hasError boolean variable set to false. The program then checks for each validation to meet the requirements one by one. If a given field is not met, then a flash message is triggerd accordingly and the hasError variable is set to true.

Saved: 4/12/2026 11:02:23 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/av847-NJIT/av847-IT202-008/blob/Milestone1/public_html/project/login.php

URL
https://github.com/av847-NJ

URL #2
https://av847-it202-008-prod.onrender.com/project/login.php

URL
https://av847-it202-008-prod.

URL #3
https://github.com/av847-NJIT/av847-IT202-008/compare/qa...Feat-MS1-BasicNav-Login

URL
https://github.com/av847-NJ

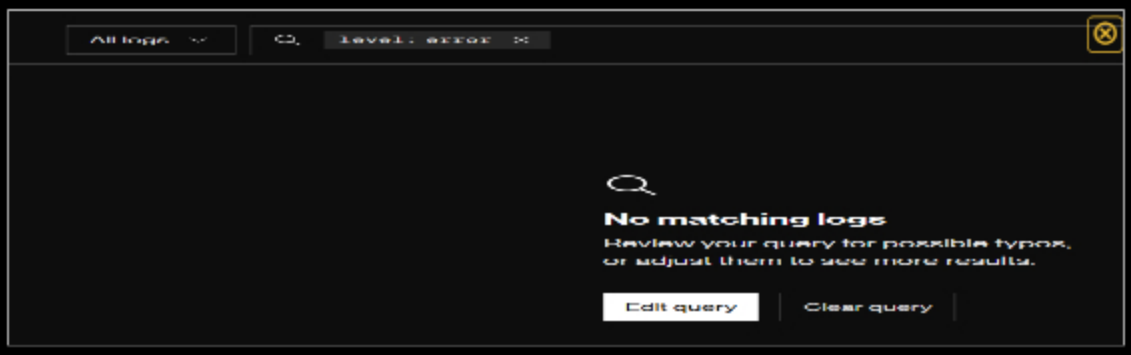
Saved: 4/12/2026 11:05:18 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

```
- From the render.com logs (Logs tab), capture the session error_log
- It should show the id, username, email, and roles
```



Error logs

Saved: 4/12/2026 11:29:37 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

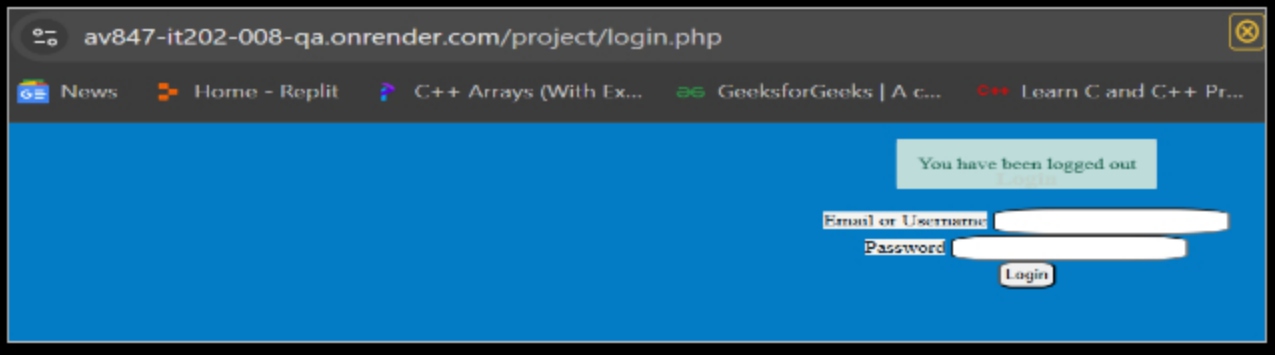
Progress: 100%

Part 1:

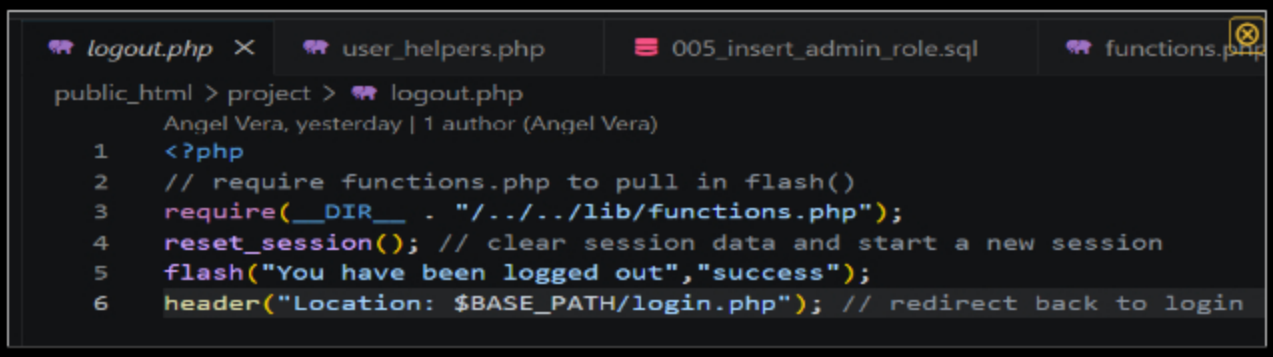
Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page



Logged out message



code of logout page

Saved: 4/12/2026 11:37:03 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/av847-NJIT/av847-IT202-008/commit/bc5b762df5d2906c09449bee316ffc0387fd4313

URL
https://github.com/av847-NJIT/av

Saved: 4/12/2026 11:37:03 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The require function allows the program to go up to root and then into the lib folder and down to the functions.php file. Then the reset_session() function is called which clears and starts a new session. From there, the flash message is triggerd and you are then redirected to the login page.

Saved: 4/12/2026 11:37:03 PM

Section #4: (2 pts.) Feature: Security Rules

Progress: 100%

Task #1 (1 pt.) - Database Tables

Progress: 100%

Details:

- From the vs code mysql tool, show both the `Roles` and `UserRoles`

```
SELECT * FROM `Roles` LIMIT 100
```

Search Results | Export | Cost: 117ms | 1 | Total 2

id int	* name varchar(20)	description varchar(100)	is_active tinyint(1)	created timestamp	modified timestamp
-1	Admin		1	2026-04-12 14:14:02	2026-04-12 14:14:02
2	Another	haha	1	2026-04-12 17:54:31	2026-04-12 17:59:11

Roles table

```
SELECT * FROM `UserRoles` LIMIT 100
```

Search Results | Export | Cost: 19ms | 1 | Total 3

id int	user_id int	role_id int	is_active tinyint(1)	created timestamp	modified timestamp
1	5	-1	1	2026-04-12 14:24:50	2026-04-12 17:52:32
2	13	-1	0	2026-04-12 18:11:02	2026-04-12 18:12:46
3	13	2	1	2026-04-12 18:11:02	2026-04-12 18:12:46

User roles table

Saved: 4/12/2026 11:38:44 PM

Task #2 (1 pt.) - Demo Security Rules

Progress: 100%

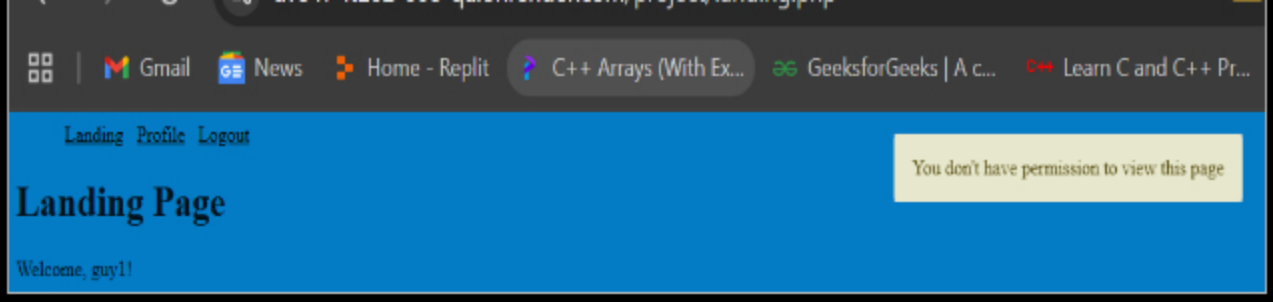
Part 1:

Progress: 100%

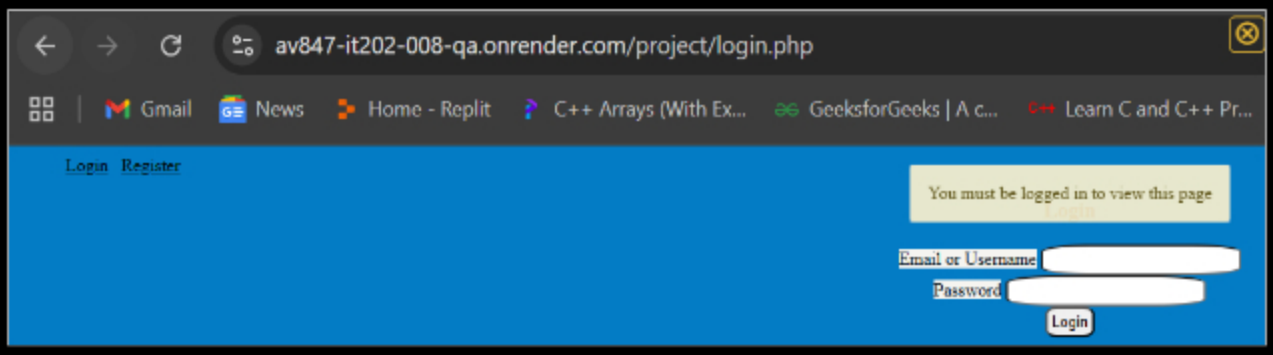
Details:

- Show the message related to needing to be logged in (i.e., try to manual
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function

av847-it202-008-ga.onrender.com/project/landing.php



Not having proper permission role to access



Needing to be logged in to access

```php
if (password_verify($password, $hash)) {

    $_SESSION["user"] = $user; // add the data to the active session
```

Login check

```php
//lookup potential roles
$stmt = $db->prepare("SELECT Roles.name FROM Roles
JOIN UserRoles on Roles.id = UserRoles.role_id
where UserRoles.user_id = :user_id and Roles.is_active = 1
and UserRoles.is_active = 1");
```

Role check

Saved: 4/12/2026 11:57:41 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/av847-NJIT/av847-IT202-008/compare/ga...Feat-MS1-UserRoles

URL
https://github.com/av847-NJIT/av

Saved: 4/12/2026 11:57:41 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

The login check code works by verifying the password and of the user and if it clears, then the data is added to the active session.

The role check essentially fetches the user's active roles after successful login. If the user had admin privileges, they are able to access the admin files which allow them to create roles, assign roles, and list roles.

Saved: 4/12/2026 11:57:41 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for

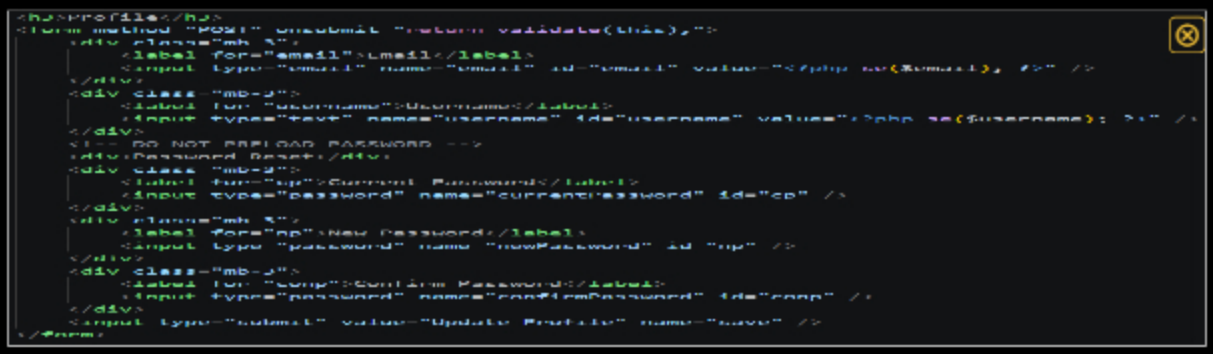
the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

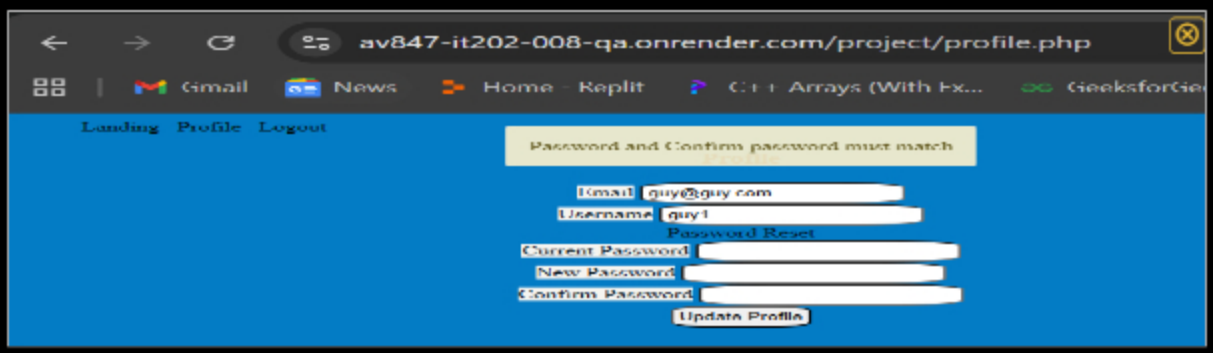
Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



HTML validation for profile page

Password to short HTML



Passwords don't match HTML

Saved: 4/13/2026 1:10:22 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validation step works by making sure the user enters the right requirements for the passwords validations to pass. If they don't, the flash messages will trigger fon the page. If they do match, the program then checks for the new password and confirm password to match. If it does not, then the proper flash message will appear telling the user that they don't.

Saved: 4/13/2026 1:10:22 AM

Task #2 (0.33 pts.) - JS Validation (validate() function)

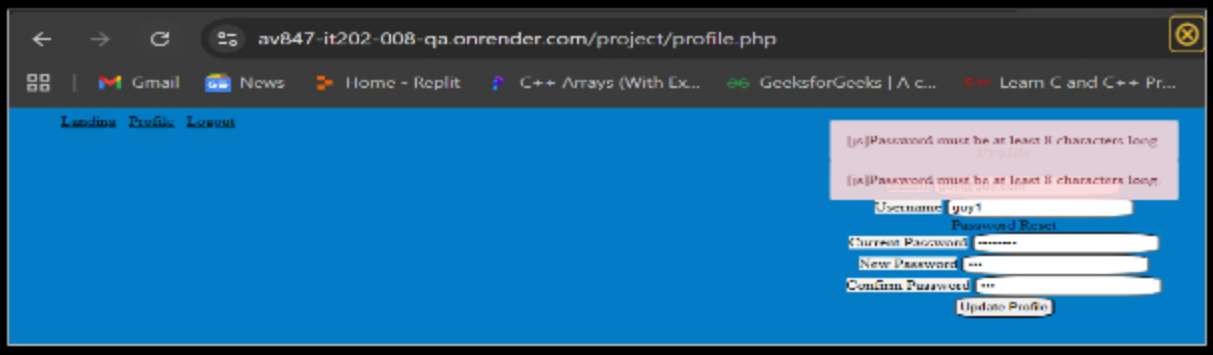
Progress: 100%

Part 1:

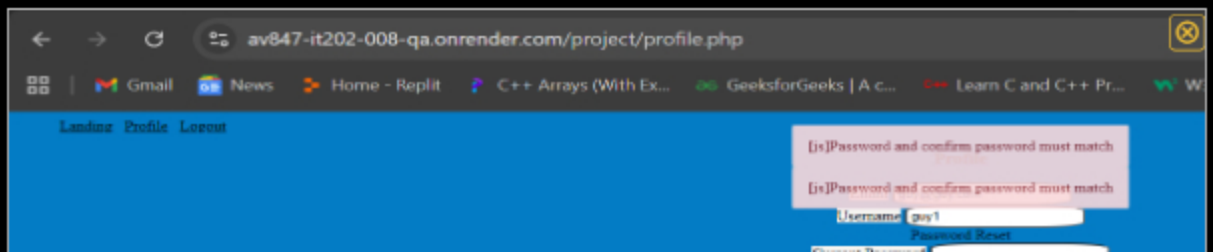
Progress: 100%

Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



Password too short JS



Password and confirm must match JS

```
<script>
    function validate(form) {
    let pw    = form.newPassword.value;
    let con   = form.confirmPassword.value;
    let currentPw = form.currentPassword.value;
    let isValid   = true;

    //example of using flash via javascript
    //find the flash container, create a new element, appendChild
    // NOTE: we'll extract the flash code to a function later
    if (currentPw && pw && con && !isValidPassword(pw)) {
        flash("Password must be at least 8 characters long.", "danger");
        isValid = false;
    }
    if (pw !== con) { // first JS validation example
        flash("Password and confirm password must match", "danger");
        isValid = false;
    }

    // returning false will prevent the form from submitting
    return isValid;
} ;  #176 156 function validate(form)        You, now • Uncommitted changes
</script>
```

JS validation in profile page

Saved: 4/13/2026 2:24:51 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

First, I established variables with the values put in. These variables included a boolean flag variable labeled isValid and it's set to true. Then, through if statement we chack to make sure the password (current, new, and confirm) all match and meet the validation criteria. If they don't, then the proper flash messages are triggered accordingly.

Saved: 4/13/2026 2:24:51 AM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

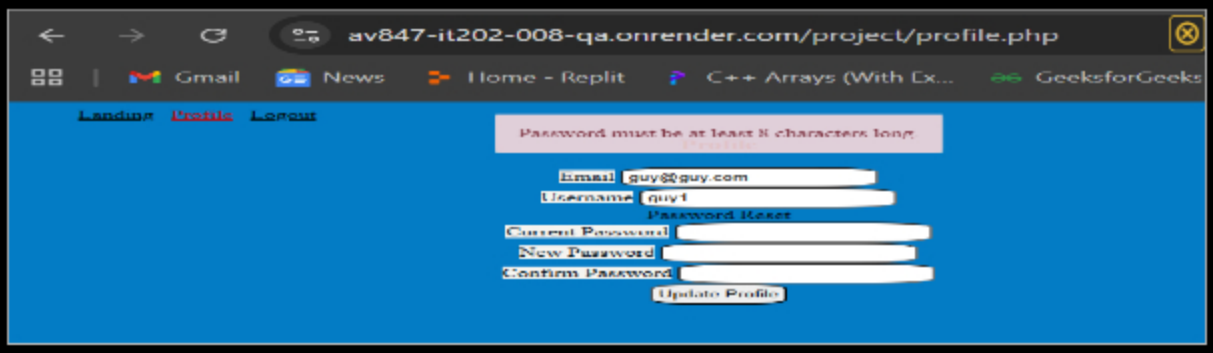
- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit

the proper file on Render.com qa after a manual deploy)

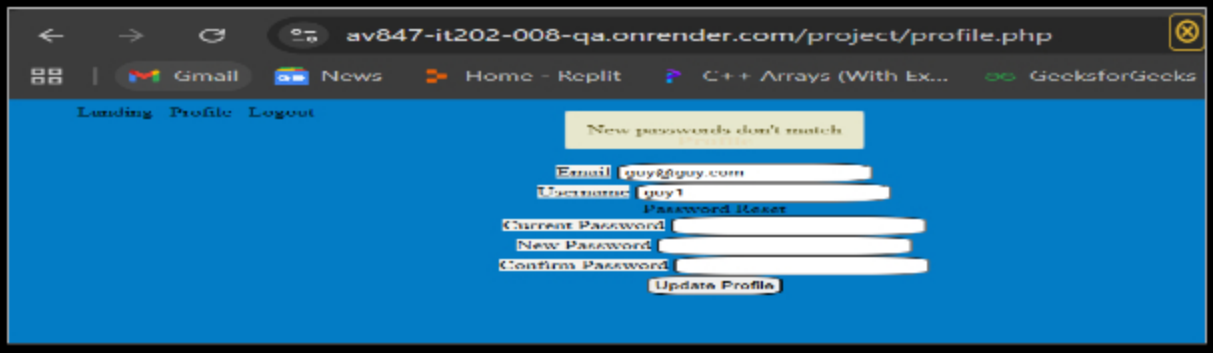
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions

```php
//check/update password
$current_password = se($_POST, "currentPassword", null, false);
$new_password = se($_POST, "newPassword", null, false);
$confirm_password = se($_POST, "confirmPassword", null, false);
// require all 3 to be set before attempting to process
$can_update = !empty($current_password) && !empty($new_password) && !empty($confirm_password);
if ($can_update) {
    // check that new matches confirm (i.e., no typos)
    if (!is_valid_confirm($new_password, $confirm_password)) {
        flash("New passwords don't match", "warning");
    } else {
        //validate current password against password rules
        $hasError = false;
        if (!is_valid_password($new_password)) {
            //echo "Password too short<br>";
            flash("Password must be at least 8 characters long.", "danger");
            $hasError = true;
        } <- #102-106 if (!is_valid_password($new_password))
```

PHP validation



Passwords too short PHP



Passwords don't match PHP

Saved: 4/13/2026 2:33:21 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validation step first verifies that it can update. This means that it checks that the current password, new password, and confirm password aren't empty. If they're not empty, then it checks if the new password and confirm password match. If they don't then the flash message pops up accordingly. Next, it checks for if the password is too short and if it is then the appropriate flash message appears for that error.

Saved: 4/13/2026 2:33:21 AM

Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/av847-NJIT/av847-IT202-008/compare/qa...Feat-MS1-UserProfile

URL
https://github.com/av847-NJ

URL #2
https://av847-it202-008-prod.onrender.com/project/profile.php

URL
https://av847-it202-008-prod.

Saved: 4/13/2026 2:36:29 AM

Section #6: (1 pt.) Misc

Progress: 100%

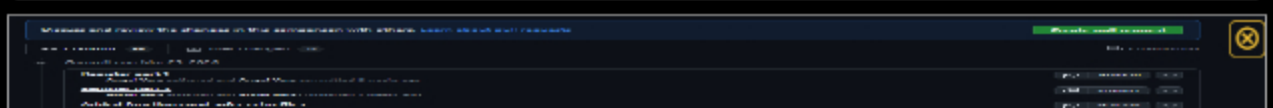
Task #1 (0.33 pts.) - Github Details

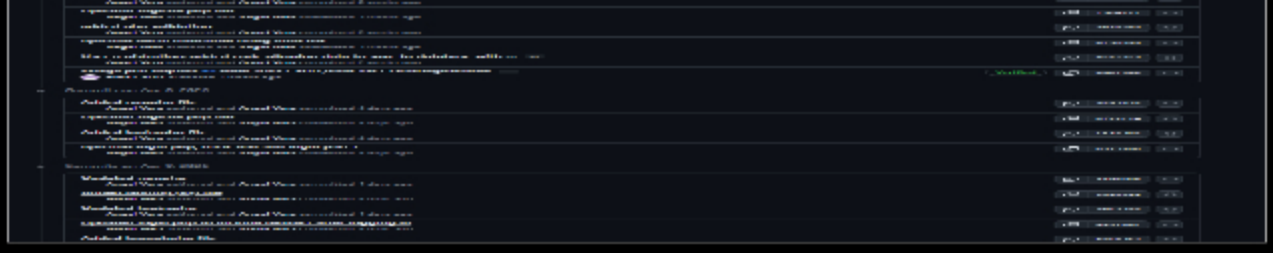
Progress: 100%

Part 1:

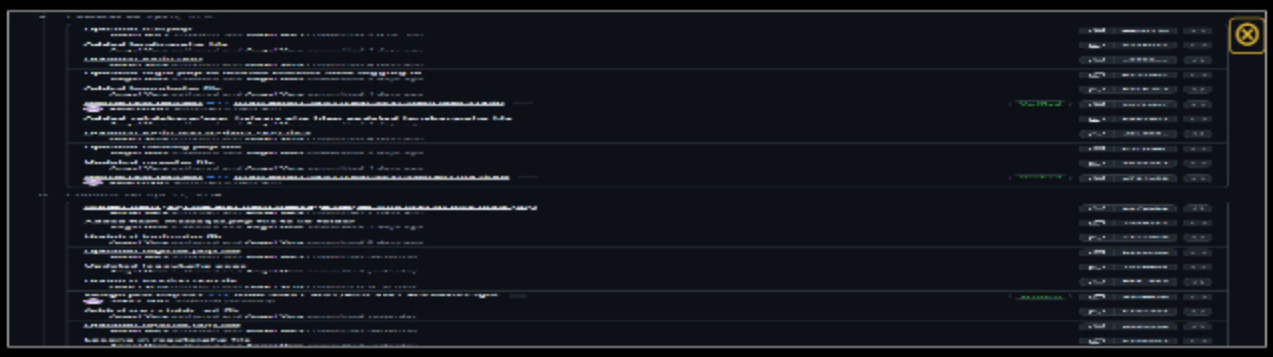
Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)





commits tab (part 1)



commits tab (part 2)

commits tab (part 3)

Saved: 4/13/2026 2:43:30 AM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

URL #1
https://github.com/av847-NJIT/av847-IT202-008/

URL
https://github.com/av847-NJIT/av

Saved: 4/13/2026 2:43:30 AM

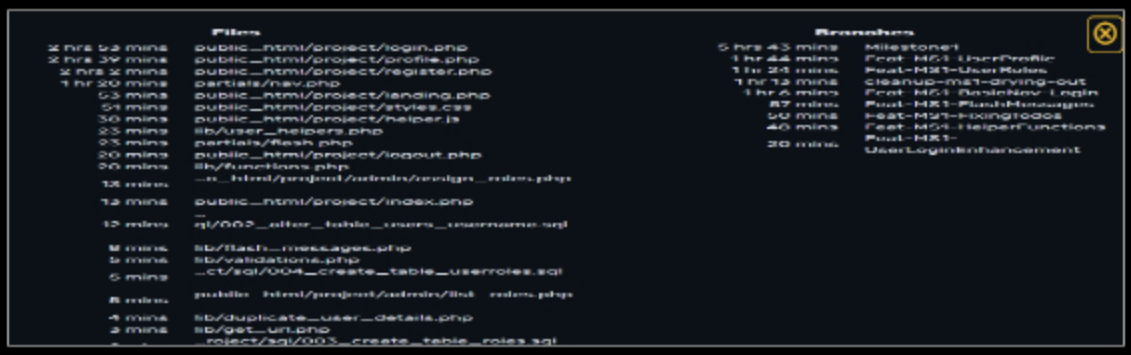
Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Waka Overview



Waka file time

Saved: 4/13/2026 2:44:58 AM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I truly learned a lot completing Milestone 1. First off, I better understand the ins and outs of full stack development as in how how to connect different files and redirect them in order to get a page to function properly. I also learned how to

better use developer tools as that was something we dearly needed to test our page. Lastly, I feel like I learned a lot about css and how to style a page to your liking.

Saved: 4/13/2026 2:48:53 AM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this assignment was copying and pasting files. It was also easy to add and commit all the different changes I was making throughout the duration of the milestone. Overall, I didn't find much easy about the assignment but I found these things to be a breeze.

Saved: 4/13/2026 2:51:09 AM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment for me was the debugging. I ran into a lot of issues getting my validations to run properly locally and on render. However, in the end I figured it out and I was able to get the validations to function.

Saved: 4/13/2026 2:52:31 AM